

OBJECT-ORIENTED PROGRAMMING (OOP)

Demo Question Paper

Time: 2 Hours	Maximum Marks: 50
Name: _____	Roll No.: _____

General Instructions:

1. Attempt all questions as directed in each section.
2. Write programs in C++ or Java.
3. Marks are shown against each question.
4. Write neat, well-indented code with suitable comments.

SECTION A: Multiple Choice Questions (10 x 1 = 10 Marks)

Attempt all questions. Choose the correct option.

1. Which of the following is NOT a pillar of OOP?
(a) Encapsulation (b) Polymorphism (c) Compilation (d) Inheritance
2. Binding data and the methods that operate on it into a single unit is called:
(a) Abstraction (b) Encapsulation (c) Inheritance (d) Overloading
3. Which keyword is used in Java to inherit a class?
(a) implements (b) inherits (c) extends (d) super
4. Method overloading is an example of:
(a) Runtime polymorphism (b) Compile-time polymorphism (c) Inheritance (d) Abstraction
5. A constructor has:
(a) A return type of void (b) The same name as its class (c) A return type of int (d) No body
6. Which of the following cannot be instantiated?
(a) A concrete class (b) A final class (c) An abstract class (d) A derived class
7. Which access specifier makes a member accessible only within its own class?
(a) public (b) protected (c) private (d) default
8. Runtime polymorphism is achieved through:
(a) Method overloading (b) Method overriding (c) Constructor overloading (d) Operator overloading
9. The `this` keyword refers to:
(a) The parent class object (b) The current object (c) A static member (d) The class itself
10. Which type of inheritance is NOT directly supported by classes in Java?
(a) Single (b) Multilevel (c) Hierarchical (d) Multiple

SECTION B: Short Answer Questions (5 x 4 = 20 Marks)

Attempt any FIVE of the following six questions.

11. Differentiate between a class and an object with a suitable example. [4]

12. What is a constructor? Explain the different types of constructors. [4]
13. Explain the difference between method overloading and method overriding. [4]
14. What is abstraction? How does it differ from encapsulation? [4]
15. Explain the static keyword and where it can be used. [4]
16. What is the difference between an abstract class and an interface? [4]

SECTION C: Long Answer Questions (2 x 10 = 20 Marks)

Attempt any TWO of the following four questions.

17. (a) Explain the four pillars of OOP with a real-world example for each. [6]
(b) List any four advantages of OOP over procedural programming. [4]
18. (a) Explain the different types of inheritance with neat diagrams. [6]
(b) Why does the diamond problem occur, and how do C++ and Java handle it? [4]
19. Design a class BankAccount with the following: [10]
- Private data members: accountNumber, holderName, balance
- A parameterized constructor
- Methods deposit(amount), withdraw(amount) and displayBalance()
- withdraw() must not allow the balance to go below zero
Then write a main() function that creates an object, deposits Rs. 5000, attempts to withdraw Rs. 7000, and displays the balance.
20. (a) Predict the output of the following code and explain your answer. [5]

```
class Animal {  
    void sound() { System.out.println("Animal makes a sound"); }  
}  
class Dog extends Animal {  
    void sound() { System.out.println("Dog barks"); }  
}  
public class Test {  
    public static void main(String[] args) {  
        Animal a = new Dog();  
        a.sound();  
    }  
}
```

- (b) Write a program demonstrating operator overloading (C++) or method overloading (Java) by adding two complex numbers. [5]

***** END OF QUESTION PAPER *****

ANSWER KEY & MARKING GUIDE (For Teacher Use Only)

Section A: Answers

1	2	3	4	5	6	7	8	9	10
c	b	c	b	b	c	c	b	b	d

Section B: Key Points Expected (4 marks each)

- Q11.** Class = blueprint/template (no memory allocated); object = instance of a class (memory allocated). Example, e.g. class Car, object myCar. (2 for definitions, 2 for example)
- Q12.** Special method with the class name, no return type, called automatically on object creation. Types: default, parameterized, copy. (1 definition, 3 for types)
- Q13.** Overloading: same name, different parameters, same class, compile-time. Overriding: same signature in subclass, runtime. (any 4 valid differences)
- Q14.** Abstraction hides implementation and shows only essential features (design level). Encapsulation wraps data and methods and restricts access (implementation level). (2 + 2)
- Q15.** Static variable, method, block, nested class. Belongs to class, not an object; shared by all instances; accessed without creating an object. (2 for concept, 2 for uses)
- Q16.** Abstract class: can have concrete methods and state, single inheritance. Interface: contract of behaviour, multiple implementation allowed. (any 4 points)

Section C: Marking Scheme (10 marks each)

- Q17.** (a) Encapsulation, abstraction, inheritance, polymorphism: 1 mark definition + 0.5 mark example each. (b) Any four advantages (reusability, modularity, security, easier maintenance, etc.): 1 mark each.
- Q18.** (a) Single, multilevel, hierarchical, multiple, hybrid: 1 mark each, with diagram credit built in. (b) Diamond problem arises when a class inherits the same base through two paths, causing ambiguity. C++: virtual inheritance. Java: no multiple class inheritance; interfaces with default methods must be resolved explicitly. (2 + 2)
- Q19.** Class with private members: 2. Constructor: 2. deposit(): 1.5. withdraw() with balance check: 2.5. displayBalance(): 1. main() with correct calls: 1. Expected output: withdrawal rejected (insufficient balance), balance remains 5000.
- Q20.** (a) Output: Dog barks (2 marks). Explanation: runtime polymorphism / dynamic method dispatch; method chosen by actual object type, not reference type (3 marks). (b) Correct class design with overloaded add/operator+: 3; correct main and output: 2.

Note: Award marks for alternative correct approaches. Give partial credit for logically correct code with minor syntax errors.